

ЛАБОРАТОРНАЯ РАБОТА №13 (2+2+2+4 БАЛЛОВ)

STL

Вариант	Задача 1	Задача 2	Задача 3	Задача 4
1	STL3Alg6	STL3Alg16	STL3Alg36	STL3Alg60
2	STL3Alg5	STL3Alg17	STL3Alg37	STL3Alg61
3	STL3Alg4	STL3Alg18	STL3Alg48	STL3Alg60
4	STL3Alg3	STL3Alg19	STL3Alg47	STL3Alg61
5	STL3Alg2	STL3Alg20	STL3Alg46	STL3Alg60
6	STL3Alg1	STL3Alg21	STL3Alg45	STL3Alg61
7	STL3Alg9	STL3Alg22	STL3Alg44	STL3Alg60
8	STL3Alg8	STL3Alg23	STL3Alg43	STL3Alg61
9	STL3Alg7	STL3Alg24	STL3Alg42	STL3Alg60
10	STL3Alg6	STL3Alg25	STL3Alg41	STL3Alg61
11	STL3Alg5	STL3Alg26	STL3Alg40	STL3Alg60
12	STL3Alg4	STL3Alg27	STL3Alg39	STL3Alg61
13	STL3Alg3	STL3Alg28	STL3Alg38	STL3Alg60
14	STL3Alg2	STL3Alg29	STL3Alg37	STL3Alg61
15	STL3Alg1	STL3Alg30	STL3Alg36	STL3Alg60

ОБОБЩЕННЫЕ АЛГОРИТМЫ

Если тип элементов контейнера в задании не указан, то предполагается, что элементами являются целые числа.

АЛГОРИТМЫ ПОИСКА

STL3Alg1. Дан вектор V . Удалить второй из элементов вектора, равных нулю. Если нулевых элементов меньше двух, то вектор не изменять. Использовать два вызова алгоритма `find` и функцию-член `erase`.

STL3Alg2. Дан дек D . Удалить последний нулевой элемент дека. Если нулевых элементов нет, то дек не изменять. Использовать алгоритм `find` с обратными итераторами и функцию-член `erase`.

Указание. Алгоритм `find` может возвращать обратный итератор, однако функция-член `erase` не позволяет его использовать для удаления элемента. Необходимо перейти от обратного итератора r к связанному с ним обычному итератору, используя функцию-член обратного итератора `r.base()`. При этом следует учитывать, что функция `r.base()` возвращает итератор, связанный с элементом, следующим за тем, с которым связан обратный итератор r . Поэтому в функции-члене `erase` следует указать одно из следующих выражений (предполагается, что r — это обратный итератор, который вернул алгоритм `find`, и этот итератор отличен от `rend()`): `--r.base()` или `(++r).base()`.

STL3Alg3. Дан список L . Удалить первый и последний нулевой элемент списка. Если нулевых элементов нет, то список не изменять, если нулевой элемент всего один, то удалить только его. Использовать два вызова алгоритма `find` и два вызова функции-члена `erase`.

Указание. Имеющуюся в классе `list` функцию-член `remove` в данном случае использовать нельзя, так как она удаляет все элементы с определенным значением. См. также указание к задаче STL3Alg2.

STL3Alg4. Дан список `L`. Удалить все элементы списка, расположенные между первым и вторым отрицательным элементом (не включая сами отрицательные элементы). Если список не содержит отрицательных элементов, то не изменять его, если отрицательный элемент всего один, то удалить все элементы, следующие за этим отрицательным элементом. Использовать два вызова алгоритма `find_if` и один вызов функции-члена `erase`.

STL3Alg5. Дан список `L`, содержащий как отрицательные, так и положительные элементы. Вставить нулевой элемент после первого отрицательного элемента и перед последним положительным элементом. Использовать два вызова алгоритма `find_if` и два вызова функции-члена `insert`.

Указание. По поводу использования алгоритма `find_if` с обратным итератором см. указание к задаче STL3Alg2.

STL3Alg6. Даны вектор `V` и список `L`; вектор `V` имеет четное количество элементов. Продублировать последний элемент списка, совпадающий с каким-либо элементом из первой половины исходного вектора. Если список не содержит требуемых элементов, то не изменять его. Использовать алгоритм `find_first_of` и функцию-член `insert` для списка.

STL3Alg7. Дан вектор `V` с четным количеством элементов. Добавить нулевой элемент перед последним элементом в первой половине вектора, совпадающим с каким-либо элементом из второй половины этого же вектора. Если вектор не содержит требуемых элементов, то не изменять его. Использовать алгоритм `find_first_of` и функцию-член `insert`.

STL3Alg8. Дано целое число $K (> 0)$ и вектор `V`, содержащий только нули и единицы. Удалить в векторе `V` последний набор из K подряд расположенных нулей (если в этом наборе имеется больше K нулей, то требуется удалить только последние K из них). Если вектор не содержит требуемого набора нулей, то не изменять его. Использовать алгоритм `search_n` и функцию-член `erase`, а также обратные итераторы.

STL3Alg9. Дано целое число $K (> 0)$ и вектор `V`. Продублировать в векторе `V` первый набор из K подряд расположенных положительных чисел, вставив после этого набора его копию (если в наборе имеется больше K положительных чисел, то лишние числа во внимание не принимаются, и дублируется только начальная часть набора, содержащая K чисел). Если вектор не содержит требуемого набора чисел, то не изменять его. Использовать алгоритм `search_n` с параметром — функциональным объектом.

БАЗОВЫЕ МОДИФИЦИРУЮЩИЕ АЛГОРИТМЫ. ИТЕРАТОРЫ ВСТАВКИ

STL3Alg16. Даны числа A и B и векторы `V1` и `V2`, каждый из которых содержит не менее 10 элементов. Заполнить первые 5 элементов каждого вектора значениями A , а последние 5 — значениями B . При преобразовании вектора `V1` использовать два вызова алгоритма `fill`, при преобразовании вектора `V2` использовать два вызова алгоритма `fill_n`.

STL3Alg17. Даны числа A и B и векторы `V1` и `V2`. Добавить в начало каждого вектора 5 элементов со значениями A , а в конец — 5 элементов со значениями B . При преобразовании

вектора `V1` использовать два вызова алгоритма `fill_n` с функциями `inserter` и `back_inserter` (данные функции возвращают итераторы вставки), при преобразовании вектора `V2` использовать два вызова функции-члена `insert`.

Замечание. Второй способ является более эффективным.

STL3Alg18. Дано число $N (> 0)$ и дек `D`, содержащий не менее $2N$ элементов. Заполнить первые N элементов дека последовательностью чисел $1, 2, \dots, N$, а последние N элементов дека — последовательностью $N, N-1, \dots, 2, 1$. Использовать два вызова алгоритма `generate` с одинаковым параметром — функциональным объектом, а также обратные итераторы.

STL3Alg19. Дано число $N (> 0)$ и дек `D`. Добавить в начало дека последовательность чисел $1, 2, \dots, N$, а в его конец — последовательность $N, N-1, \dots, 2, 1$. Использовать два вызова алгоритма `generate_n` с одинаковым параметром — функциональным объектом, а также итераторы вставки, возвращаемые функциями `front_inserter` и `back_inserter`.

STL3Alg20. Даны списки `L1` и `L2`, каждый из которых содержит четное количество элементов. Поменять местами первую и вторую половину каждого списка (например, список $1, 2, 3, 4$ должен быть преобразован следующим образом: $3, 4, 1, 2$). Для первого списка использовать алгоритм `swar_ranges`, для второго — алгоритм `rotate`. Использовать также функцию `advance`.

STL3Alg21. Дано число $K (0 < K < 10)$ и списки `L1` и `L2`, каждый из которых содержит не менее 10 элементов. Выполнить для списка `L1` циклический сдвиг элементов вправо на K позиций, а для списка `L2` — циклический сдвиг влево на K позиций. Использовать алгоритм `rotate` и функцию `advance`.

STL3Alg22. Дано число $K (0 < K < 5)$ и список `L`, содержащий не менее 10 элементов. Набор из первых 5 элементов списка скопировать в конец списка, выполнив для этой копии циклический сдвиг на K позиций вправо, а набор из последних 5 элементов исходного списка скопировать в начало списка, выполнив для этой копии циклический сдвиг на K позиций влево. Использовать два вызова алгоритма `rotate_copy` и итераторы вставки, а также функцию `advance`.

STL3Alg23. Даны списки `L1` и `L2`, у каждого из которых количество элементов делится на 4. В первом списке инвертировать (расположить в обратном порядке) первую половину элементов, во втором списке — вторую половину. Для первого списка использовать алгоритм `swar_ranges` и обратные итераторы, для второго — алгоритм `reverse`. Использовать также функцию `advance`.

STL3Alg24. Даны списки `L1` и `L2`, каждый из которых содержит четное количество элементов. В каждом списке продублировать первую половину, добавив ее элементы в конец списка в обратном порядке. Для первого списка использовать алгоритм `reverse_copy` и итератор вставки, для второго — функцию-член `insert` и обратные итераторы. Использовать также функцию `advance`.

Замечание. Второй способ является более эффективным.

STL3Alg25. Дан вектор `V` с четным количеством элементов. В первой половине исходного вектора заменить все отрицательные числа на -1 , а во второй — все положительные числа на 1 . Использовать два вызова алгоритма `replace_if` с различными параметрами — функциональными объектами.

STL3Alg26. Дан список *L* с четным количеством элементов. Скопировать в конец списка все элементы, расположенные в его первой половине, заменив при этом отрицательные элементы на нули и расположив скопированные элементы в обратном порядке. Использовать алгоритм `replace_copy_if`, итератор вставки и обратные итераторы, а также функцию `advance`.

STL3Alg27. Дан дек *D* с четным количеством элементов. Решить для него задачу STL3Alg26. Поскольку операция вставки делает все итераторы дека недействительными, для решения задачи использовать вспомогательный дек *D0*. Инициализировать дек *D0* первой половиной элементов дека *D*, после чего применить алгоритм `replace_copy_if`, используя дек *D0* как источник, а дек *D* как приемник данных.

STL3Alg28. Дан список *L* с четным количеством элементов. Скопировать в начало списка все положительные элементы, расположенные в его второй половине, сохранив для них исходный порядок следования. Использовать алгоритм `remove_copy_if` и итератор вставки, а также функцию `advance`.

STL3Alg29. Дан вектор *V* с четным количеством элементов. Решить для него задачу STL3Alg28. Поскольку операция вставки в начало вектора делает все его итераторы недействительными, для решения задачи использовать вспомогательный вектор *V0*. Инициализировать вектор *V0* второй половиной элементов вектора *V*, после чего применить алгоритм `remove_copy_if`, используя вектор *V0* как источник, а вектор *V* как приемник данных.

STL3Alg30. Дан вектор *V* с четным количеством элементов. Удалить все нулевые элементы, расположенные во второй половине исходного вектора. Использовать алгоритм `remove` и функцию-член `erase`.

Указание. Алгоритм `remove` и другие алгоритмы, связанные с удалением элементов, не удаляют требуемые элементы, а лишь перемещают их в конец указанного диапазона и возвращают итератор на начало диапазона с перемещенными элементами. Для фактического удаления элементов после выполнения алгоритма необходимо вызвать функцию-член `erase`.

СОРТИРОВКА И СЛИЯНИЕ

STL3Alg36. Дан вектор *V* с нечетным количеством элементов $N (\geq 3)$. Определить значения трех средних элементов вектора после того, как вектор будет отсортирован (по возрастанию), и вывести их в порядке возрастания. Использовать алгоритмы `nth_element` (для нахождения центрального элемента), `max_element` (для нахождения элемента, предшествующего центральному) и `min_element` (для нахождения элемента, следующего за центральным).

STL3Alg37. Дан вектор *V*, содержащий не менее 3 элементов. Определить значения трех начальных элементов вектора после того, как вектор будет отсортирован (по возрастанию), и вывести их в порядке возрастания. Использовать один вызов алгоритма `partial_sort` и алгоритм `copy` для вывода требуемых элементов.

STL3Alg38. Дан вектор *V*, содержащий не менее 3 элементов. Определить значения трех конечных элементов вектора после того, как вектор будет отсортирован (по возрастанию) и вывести их в порядке убывания. Использовать один вызов алгоритма `partial_sort` с параметром — функциональным объектом `greater` и алгоритм `copy` для вывода требуемых элементов.

STL3Alg39. Дан вектор *V* с четным количеством элементов. Добавить в его конец первую половину элементов, которые содержал бы отсортированный по возрастанию вариант исходного вектора. Использовать функцию-член `insert` для увеличения количества элементов вектора на требуемую величину и алгоритм `partial_sort_copy`.

STL3Alg40. Дан список *L*. Определить количество четных и нечетных чисел в исходном списке (вначале вывести количество четных, затем количество нечетных чисел). Использовать алгоритм `partition` и два вызова функции `distance` для итераторов.

STL3Alg41. Дан список *L*. Перегруппировать элементы списка, расположив в нем вначале положительные, а затем неположительные элементы (порядок расположения элементов в каждой группе должен совпадать с исходным). Использовать алгоритм `stable_partition`.

STL3Alg42. Дан список *L*. Перегруппировать элементы списка, расположив в нем вначале отрицательные, затем нулевые, а затем положительные элементы (порядок расположения элементов в каждой группе должен совпадать с исходным). Использовать два вызова алгоритма `stable_partition`.

STL3Alg43. Дан список *L* с четным количеством элементов. Перегруппировать элементы списка, расположив в нем вначале все четные элементы из первой половины исходного списка, затем все нечетные элементы, а затем — все четные элементы из второй половины списка (порядок расположения элементов в каждой группе должен совпадать с исходным). Использовать два вызова алгоритма `stable_partition`.

STL3Alg44. Дан дек *D*. Решить для него задачу STL3Alg42, используя один вызов алгоритма `stable_sort` с параметром — функциональным объектом.

STL3Alg45. Дан дек *D*, элементами которого являются английские слова. Отсортировать его элементы по возрастанию их длин, а элементы одинаковой длины — в алфавитном порядке. Использовать алгоритм `sort` для сортировки по алфавиту и затем алгоритм `stable_sort` с параметром — функциональным объектом для сортировки по длине. Выводить новое содержимое дека после применения каждого алгоритма.

STL3Alg46. Дан дек *D*, элементами которого являются английские слова. Отсортировать его элементы по убыванию их длин, а элементы одинаковой длины — в алфавитном порядке. Использовать единственный вызов алгоритма `sort` с параметром — функциональным объектом, включающим как сравнение строк, так и сравнение их длин.

STL3Alg47. Дан вектор *V* с четным количеством элементов. Известно, что первая половина вектора уже отсортирована по возрастанию. Отсортировать все элементы вектора по возрастанию, выполнив вначале сортировку его второй половины алгоритмом `sort`, а затем слияние обеих половин алгоритмом `inplace_merge`. Выводить новое содержимое вектора *V* после применения каждого алгоритма.

STL3Alg48. Дан вектор *V*, количество элементов которого делится на 3. Известно, что каждая треть вектора уже упорядочена по возрастанию. Решить для исходного вектора задачу STL3Alg42, используя два вызова алгоритма `inplace_merge` с одним и тем же параметром — функциональным объектом.

ЧИСЛЕННЫЕ АЛГОРИТМЫ

STL3Alg60. Дан список L . Получить вектор V вещественных чисел, содержащий значения среднего арифметического для всех пар соседних элементов исходного списка (количество элементов вектора V должно быть на 1 меньше количества элементов списка L). Например, для исходного списка 1, 3, 4, 6 полученный вектор должен содержать значения 2.0, 3.5, 5.0. Использовать алгоритм `adjacent_difference` с итератором вставки и функциональным объектом, а также функцию-член `erase` для вектора V .

STL3Alg61. Дан список L , элементами которого являются английские слова. Получить дек D со строковыми элементами, каждый из которых строится по паре соседних элементов исходного списка L следующим образом: последняя буква правого элемента пары приписывается справа к первой букве левого элемента пары. Количество элементов дека D должно быть на 1 меньше количества элементов списка L . Например, для исходного списка ABC, DEF, KLM, XYZ полученный дек должен содержать строки AF, DM, KZ. Использовать алгоритм `adjacent_difference` с итератором вставки и функциональным объектом, а также функцию-член `erase` для дека D .